MSO API Proxy

Guía técnica y de operación

API | Arquitectura | Configuración | Seguridad | Despliegue

Servicio	mso-api-proxy
Consumidor de referencia	Dashboard Intelfon
Runtime	Node.js 18 o posterior
Dependencias runtime	Ninguna (módulos nativos de Node.js)

Credenciales: esta guía documenta el nombre de la variable y usa marcadores. No contiene ni debe contener valores reales de claves API.

Contenido

01. 1. Resumen ejecutivo

02. 2. Arquitectura y flujo de solicitud

03. 3. Requisitos e instalación

04. 4. Configuración de entorno

05. 5. Contrato HTTP

06. 6. Respuestas y diagnóstico

07. 7. Caché y concurrencia

08. 8. CORS

09. 9. Seguridad y manejo de claves

10. 10. Despliegue y operación

11. 11. Referencia de implementación

1. Resumen ejecutivo

Este servicio expone un proxy HTTP pequeño delante de la API MSO. El navegador consulta el proxy; el proxy agrega la credencial `x-api-key` y envía la solicitud a MSO. Así se evita distribuir la clave upstream al cliente.

El servicio:

- Acepta solicitudes `GET` y preflight CORS `OPTIONS`.
- Publica dos prefijos de negocio que se traducen a prefijos upstream.
- Reenvía el sufijo de ruta y los parámetros de consulta.
- Almacena en caché las respuestas upstream con estado `200` durante un TTL configurable.
- Agrupa solicitudes idénticas que llegan simultáneamente.
- No requiere paquetes npm adicionales.

Límites conocidos

El repositorio no especifica los endpoints concretos debajo de los prefijos, los parámetros de negocio ni los esquemas JSON de MSO. Los sufijos mostrados en esta guía son ejemplos de reenvío, no una afirmación de que el proveedor publique esos recursos. Confirma los nombres y contratos con la documentación vigente de MSO.

2. Arquitectura y flujo de solicitud

1. El cliente envía una solicitud al proxy, por ejemplo `GET /api/mso/simcards/<recurso>`. 2. El proxy valida que el método sea `GET`, traduce el prefijo local y conserva el sufijo. 3. El proxy elimina el parámetro reservado `refresh`, si está presente. 4. Si corresponde, responde desde la caché; si no, realiza una solicitud HTTPS a MSO. 5. El proxy envía `x-api-key` y `Accept: application/json` al upstream. 6. La respuesta upstream conserva su estado HTTP y cuerpo. El proxy agrega encabezados de caché y CORS.

La clave no forma parte del contrato que consume el navegador. No se envía en query string ni se devuelve en las respuestas del proxy.

3. Requisitos e instalación

- Node.js `>=18`.
- Acceso de red HTTPS al host de MSO.
- Una clave MSO vigente, configurada en el entorno del servidor.

Clona o descarga el proyecto, copia `.env.example` a `.env`, completa `MSO_API_KEY` y ejecuta:

```
npm start
```

No es necesario ejecutar `npm install`: el proyecto no declara dependencias runtime.

Configuración local

`.env` es un archivo de desarrollo local que el servidor lee al iniciar. Mantenlo fuera del control de versiones; el repositorio ya excluye `.env` en `.gitignore`.

Ejemplo de `.env` (el valor es un marcador, no una clave real):

```
MSO_API_KEY=REEMPLAZAR_CON_TU_CLAVE
MSO_API_BASE_URL=https://kpi.red.com.sv/api
PORT=3000
HOST=0.0.0.0
CACHE_TTL_MS=60000
MSO_API_TIMEOUT_MS=30000
ALLOW_ORIGIN=http://localhost:5500
```

El cargador admite asignaciones simples `NOMBRE=valor` y quita comillas simples o dobles exteriores. Para producción, define las variables en el entorno o gestor de secretos del proveedor, no en un archivo incluido en la imagen o el repositorio.

Variable temporal en PowerShell

```
$env:MSO_API_KEY = "REEMPLAZAR_CON_TU_CLAVE"
npm start
```

La variable de entorno de PowerShell existe solo en esa sesión y se hereda por el proceso iniciado desde ella.

4. Configuración de entorno

Variable	Requerida	Predeterminado	Descripción
`MSO_API_KEY`	Sí	Sin valor	Clave enviada en `x-api-key` al upstream. El proceso termina al iniciar si falta.
`MSO_API_BASE_URL`	No	`https://kpi.red.com.sv/api`	Base URL upstream. El servidor agrega el prefijo de ruta traducido.
`PORT`	No	`3000`	Puerto del servidor HTTP.
`HOST`	No	`0.0.0.0`	Interfaz de escucha del servidor.
`CACHE_TTL_MS`	No	`60000`	TTL de caché, en milisegundos.
`MSO_API_TIMEOUT_MS`	No	`30000`	Límite de espera para la solicitud upstream, en milisegundos.
`ALLOW_ORIGIN`	No	`*`	`*` o lista de orígenes CORS separados por coma.

Los valores de ejemplo no sustituyen la validación de configuración propia del entorno. Mantén el TTL, timeout y orígenes autorizados alineados con los requisitos de despliegue.

5. Contrato HTTP

Rutas públicas

Metodo	Ruta	Comportamiento
`GET`	`/`	Devuelve el estado del servicio.
`GET`	`/api/health`	Devuelve el estado y metadatos de caché.
`GET`	`/api/mso/simcards` y subrutas	Reenvía al prefijo MSO `/dashboard-simcards`.
`GET`	`/api/mso/reclamos` y subrutas	Reenvía al prefijo MSO `/dashboard-reclamos-m2m`.
`OPTIONS`	Cualquier ruta	Responde al preflight CORS con estado `204`.
Otros métodos	Cualquier ruta	Responde `405` con `{"ok":false,"error":"Solo GET"}`.

Las rutas de proxy preservan el sufijo. Por ejemplo:

Solicitud al proxy	Ruta enviada a MSO
`/api/mso/simcards`	`/dashboard-simcards`
`/api/mso/simcards/<sufijo>`	`/dashboard-simcards/<sufijo>`
`/api/mso/reclamos/<sufijo>`	`/dashboard-reclamos-m2m/<sufijo>`

El sufijo `<sufijo>` representa una ruta documentada por MSO. Reemplázalo con el recurso real autorizado por el proveedor.

Ejemplos de consumo

Consulta la raíz del prefijo de tarjetas SIM:

```
curl -i "http://localhost:3000/api/mso/simcards"
```

Consulta un recurso definido por MSO, conservando ruta y parámetros:

```
curl -i "http://localhost:3000/api/mso/simcards/<ruta-documentada>?<parametro>=<valor>"
```

Consulta un recurso de reclamos:

```
curl -i "http://localhost:3000/api/mso/reclamos/<ruta-documentada>?<parametro>=<valor>"
```

El cliente no agrega la clave MSO; el proxy la incorpora. Codifica correctamente los valores de query string según las reglas HTTP.

Forzar actualización

```
curl -i "http://localhost:3000/api/mso/simcards/<ruta-
documentada>?<parametro>=<valor>&refresh=1"
```

La presencia de `refresh` omite la lectura de caché y ese parámetro no se reenvía a MSO. No es un control de permisos ni invalida otras entradas de caché. Una solicitud forzada aún puede compartir una solicitud upstream idéntica que ya esté en curso.

Health check

```
curl -i "http://localhost:3000/api/health"
```

Respuesta representativa:

```
{
  "ok": true,
  "service": "mso-api-proxy",
  "source": "https://kpi.red.com.sv/api",
  "cache": {
    "entries": 0,
    "ttl_ms": 60000
  },
  "now": "<timestamp ISO-8601>"
}
```

`now` es dinámico. La ruta no comprueba que la clave funcione ni realiza una llamada de salud al upstream.

6. Respuestas y diagnóstico

En una respuesta de proxy se mantiene el estado HTTP upstream y el cuerpo recibido. El proxy declara `Content-Type: application/json`; valida la respuesta del proveedor por separado si la aplicación requiere garantizar un esquema JSON.

Estado / encabezado	Significado
`X-Cache: HIT`	Respuesta `200` servida desde caché.
`X-Cache: MISS`	Solicitud upstream nueva; puede cachearse si devuelve `200`.
`X-Cache: COALESCED`	Solicitud idéntica reutilizó una llamada upstream concurrente.
`X-Cache: ERROR`	Error de conexión o timeout upstream; el proxy responde `502`.
`404`	Ruta local desconocida.
`405`	Metodo distinto de `GET` o `OPTIONS`.
`502`	Error de red o timeout al consultar MSO; cuerpo JSON con `ok: false` y `error`.

Las respuestas locales incluyen `Cache-Control: no-store` en la ruta de proxy. Los códigos upstream como `401`, `403` o `5xx` se devuelven tal cual y no se guardan en caché; revisa la respuesta de MSO y las credenciales/configuración del servicio.

7. Caché y concurrencia

- La caché reside en memoria del proceso; un reinicio la vacía.
- Solo se almacenan respuestas upstream con estado `200`.
- Cada entrada vence tras `CACHE_TTL_MS`.
- La clave de caché combina la ruta upstream y el query string serializado.
- Consultas con distintos parámetros forman entradas diferentes.
- Peticiones idénticas concurrentes comparten una única solicitud upstream.
- `refresh` omite la lectura de caché, se elimina antes del reenvío y no evita la coalescencia de solicitudes en curso.
- `X-Cache` informa `HIT`, `MISS`, `COALESCED` o `ERROR`.

Esta implementación no es una caché distribuida. Con varias instancias, cada una mantiene su propia caché y puede hacer solicitudes upstream independientes.

8. CORS

`ALLOW_ORIGIN` acepta `*` o una lista separada por comas, por ejemplo:

```
ALLOW_ORIGIN=https://dashboard.example.com,http://localhost:5500
```

En desarrollo, el valor predeterminado `*` permite solicitudes cross-origin desde cualquier origen. En producción, configura solo los orígenes del dashboard. El servidor anuncia los métodos `GET, OPTIONS` y permite el encabezado `Content-Type`; no habilita credenciales CORS. CORS es una política del navegador y no reemplaza autenticación/autorización del servicio.

9. Seguridad y manejo de claves

1. Proporciona la clave al proceso mediante un gestor de secretos o variable protegida. 2. No incluyas claves reales en el repositorio, PDF, ejemplos, logs, URLs, frontend, tickets o capturas. 3. Mantén `.env` local fuera de Git; `.env.example` contiene solo nombres y marcadores sin secreto. 4. Restringe `ALLOW_ORIGIN` al dominio del dashboard en despliegues públicos. 5. Expón el servicio solo detrás de HTTPS y controles de red/autorización adecuados para el entorno. 6. Rota inmediatamente cualquier clave que se haya compartido o publicado accidentalmente.

El proxy no implementa autenticación para sus propios consumidores. La protección de acceso debe proveerse en la red o en una capa gateway apropiada al despliegue.

10. Despliegue y operación

1. Selecciona un entorno Node.js 18 o superior. 2. Configura `MSO_API_KEY` en el gestor de secretos del proveedor. 3. Configura, según el entorno, `PORT`, `HOST`, `CACHE_TTL_MS`, `MSO_API_TIMEOUT_MS` y `ALLOW_ORIGIN`. 4. Inicia con `npm start` o `node server.js`. 5. Configura el health check del proveedor en `GET /api/health`. 6. Verifica salida HTTPS al host configurado en `MSO_API_BASE_URL`. 7. Prueba primero los prefijos con recursos confirmados por MSO.

El proveedor debe enrutar el puerto de escucha al servicio. Usa logs de proceso para confirmar arranque; el servicio informa la URL local, la fuente, el TTL y los orígenes configurados, pero no imprime el valor de la clave.

11. Referencia de implementación

Elemento	Definición
Archivo de arranque	`server.js`
Comando de inicio	`npm start`
Clave upstream	Encabezado `x-api-key`
Formato solicitado al upstream	`Accept: application/json`
Transporte upstream	HTTPS
Métodos de negocio	`GET`
Prefijo de tarjetas SIM	`/api/mso/simcards` -> `/dashboard-simcards`
Prefijo de reclamos	`/api/mso/reclamos` -> `/dashboard-reclamos-m2m`

Contratos upstream pendientes

Para documentar el uso funcional completo de MSO con mayor precisión, solicita o agrega una fuente oficial para:

- Rutas concretas disponibles debajo de cada prefijo.
- Parámetros, formatos, filtros y paginación.
- Esquemas de solicitud/respuesta y ejemplos anonimizados.
- Políticas de autenticación, límites de consumo y códigos de error.

No se inventan esos contratos en esta guía porque no están definidos en el código fuente del proxy.